4.2: Communication Protocols

Introduction to Communication Protocols

Microcontrollers like the ESP32 need to communicate with various sensors, actuators, and other devices to perform their functions. This communication happens through **communication protocols**, which are sets of rules that govern how data is exchanged between two or more devices. Understanding these protocols is fundamental for anyone working with embedded systems and IoT devices.

This module will introduce three essential communication protocols commonly used with the ESP32: **Serial (UART)**, **I2C (Inter-Integrated Circuit)**, and **SPI (Serial Peripheral Interface)**. We will explore their principles, applications, and how to implement them with the ESP32, focusing on their use for debugging, connecting sensors, and high-speed peripherals.

1. Serial Communication (UART)

UART (Universal Asynchronous Receiver-Transmitter) is one of the oldest and most widely used forms of serial communication. It allows two devices to communicate directly with each other, sending data one bit at a time over a single wire (or two wires for full-duplex communication).

Principles of UART

UART communication involves two main lines: **TX (Transmit)** and **RX (Receive)**. Data is transmitted asynchronously, meaning there is no shared clock signal between the sender and receiver. Instead, both devices must agree on a common baud rate (data transfer speed) and data framing parameters (start bit, data bits, parity bit, stop bits).

- **Start Bit:** A logic LOW signal that indicates the start of a new data frame.
- **Data Bits:** Typically 5 to 9 bits of actual data. 8 data bits are most common.
- **Parity Bit (Optional):** Used for error checking. It can be even or odd parity.

- **Stop Bit(s):** A logic HIGH signal (1 or 2 bits) that indicates the end of the data frame.

Applications of UART with ESP32

UART is primarily used with the ESP32 for:

- **Debugging and Console Output:** The most common use case. The ESP32's built-in USB-to-Serial converter allows you to connect it to your computer and view debug messages, sensor readings, or program output through a serial monitor (e.g., in the Arduino IDE or PlatformIO).
- **Communication with GPS Modules:** Many GPS modules use UART to send location data to the microcontroller.
- **Interfacing with Bluetooth Modules:** Older Bluetooth modules (like HC-05/06) often communicate via UART.
- **Connecting to other Microcontrollers:** Simple, direct communication between two microcontrollers.

Example: ESP32 Serial Debugging

Here's a simple Arduino-style example demonstrating how to use UART for debugging on the ESP32.

```
void setup() {
  Serial.begin(115200); // Initialize serial communication at 115200 baud
rate
  Serial.println("ESP32 Booted Up!");
}

void loop() {
  Serial.println("Hello from ESP32!");
  delay(1000); // Wait for 1 second
}
```

2. I2C Communication (Inter-Integrated Circuit)

I2C (Inter-Integrated Circuit), also known as TWI (Two-Wire Interface), is a synchronous, multi-master, multi-slave serial computer bus invented by Philips Semiconductor (now NXP Semiconductors). It is widely used for connecting low-speed peripheral ICs to processors and microcontrollers.

Principles of I2C

I2C uses only two bidirectional open-drain lines:

- **SDA (Serial Data Line):** Carries the data.
- **SCL (Serial Clock Line):** Carries the clock signal, synchronized by the master device.

Both lines require pull-up resistors to ensure proper operation. Each device on the I2C bus has a unique 7-bit or 10-bit address. Communication occurs in a master-slave configuration, where the master initiates communication and generates the clock signal. Multiple masters and slaves can exist on the same bus.

Applications of I2C with ESP32

I2C is ideal for connecting multiple sensors and other low-speed devices to the ESP32, such as:

- **Environmental Sensors:** BME280 (temperature, humidity, pressure), DHT12 (temperature, humidity).
- **Motion Sensors:** MPU6050 (accelerometer, gyroscope).
- **Real-Time Clocks (RTC):** DS3231.
- **OLED/LCD Displays:** Small character or graphic displays.
- **EEPROM Memory:** External non-volatile memory.

Example: Reading Temperature from a BME280 Sensor via I2C

This example assumes you have the Adafruit BME280 library installed.

```
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BME280.h>

#define BME_SCK 13
#define BME_MISO 12
#define BME_MOSI 11
#define BME_CS 10

#define SEALEVELPRESSURE_HPA (1013.25)

Adafruit_BME280 bme; // I2C

void setup() {
  Serial.begin(115200);
  Serial.println(F("BME280 test"));

  bool status;

  // Default settings
  status = bme.begin(0x76);
  if (!status) {
    Serial.println("Could not find a valid BME280 sensor, check wiring!");
    while (1);
  }
  Serial.println("-- BME280 found --");
}

void loop() {
  Serial.print("Temperature = ");
  Serial.print(bme.readTemperature());
  Serial.println(" *C");

  Serial.print("Humidity = ");
  Serial.print(bme.readHumidity());
  Serial.println(" %");

  Serial.print("Pressure = ");
  Serial.print(bme.readPressure() / 100.0F);
  Serial.println(" hPa");

  Serial.println();
  delay(2000);
}
```

3. SPI Communication (Serial Peripheral Interface)

SPI (Serial Peripheral Interface) is a synchronous serial communication interface specification used for short-distance communication, primarily in embedded systems. It operates in full-duplex mode, meaning data can be sent and received simultaneously.

Principles of SPI

SPI uses four logic signals:

- **SCLK (Serial Clock):** Generated by the master to synchronize data transfer.
- **MOSI (Master Out Slave In):** Data line from the master to the slave.
- **MISO (Master In Slave Out):** Data line from the slave to the master.
- **CS/SS (Chip Select/Slave Select):** An active-low signal used by the master to select a specific slave device when multiple slaves are present on the bus. Each slave needs its own CS line.

SPI is faster than I2C and UART because it uses a dedicated data line for each direction and a separate clock line, allowing for higher data rates and full-duplex operation. It also doesn't require unique addresses for each slave; instead, slaves are selected via their dedicated CS line.

Applications of SPI with ESP32

SPI is preferred for high-speed peripherals and devices that require fast data transfer, such as:

- **SD Card Modules:** For data logging and storage.
- **TFT/LCD Displays:** High-resolution graphic displays.
- **Ethernet Controllers:** ENC28J60.
- **Flash Memory:** External non-volatile memory for storing large amounts of data.
- **Radio Transceivers:** NRF24L01 (2.4 GHz wireless communication).

Example: Interfacing with an SD Card Module via SPI

This example demonstrates initializing an SD card and writing data to a file. It assumes you have the SD library installed.

```
#include <SPI.h>
#include <SD.h>

File myFile;

void setup() {
  Serial.begin(115200);
  while (!Serial) {
    ;
  }

  Serial.print("Initializing SD card...");

  if (!SD.begin(5)) { // Chip Select pin for ESP32 is often GPIO 5
    Serial.println("Card failed, or not present");
    while (1);
  }
  Serial.println("card initialized.");

  // open the file. note that only one file can be open at a time,
  // so you have to close this one before opening another.
  myFile = SD.open("test.txt", FILE_WRITE);

  // if the file opened okay, write to it:
  if (myFile) {
    Serial.print("Writing to test.txt...");
    myFile.println("testing 1, 2, 3.");
    // close the file:
    myFile.close();
    Serial.println("done.");
  } else {
    // if the file didn't open, print an error:
    Serial.println("error opening test.txt");
  }

  // re-open the file for reading:
  myFile = SD.open("test.txt");
  if (myFile) {
    Serial.println("test.txt:");

    // read from the file until there's nothing else in it:
    while (myFile.available()) {
      Serial.write(myFile.read());
    }
    // close the file:
```

```
    myFile.close();
  } else {
    // if the file didn't open, print an error:
    Serial.println("error opening test.txt");
  }
}

void loop() {
  // nothing happens after setup
}
```

Comparison of Communication Protocols

Feature	UART	I2C	SPI
Lines Used	2 (TX, RX)	2 (SDA, SCL)	4 (MOSI, MISO, SCLK, CS)
Clock	Asynchronous (no shared clock)	Synchronous (shared clock, master-driven)	Synchronous (shared clock, master-driven)
Speed	Moderate (up to ~1 Mbps)	Moderate (up to ~3.4 Mbps)	Fast (up to tens of Mbps)
Devices	2 (point-to-point)	Multi-master, multi-slave	Single master, multi-slave
Addressing	No (direct connection)	7-bit or 10-bit addresses	Chip Select (CS) line for each slave
Complexity	Simple	Moderate	Moderate
Applications	Debugging, GPS, Bluetooth (legacy)	Sensors, RTC, small displays, EEPROM	SD cards, TFT displays, Flash memory, Radio

Practice Problems

1. **UART:** You are building an ESP32-based weather station. You want to send the current temperature and humidity readings to your computer's serial monitor every 5 seconds for debugging. Write the Arduino-style code to achieve this,

assuming you have temperature and humidity values available in `float temperature` and `float humidity` variables.

- **Solution:**

```
void setup() {
  Serial.begin(115200);
  Serial.println("Weather Station Debugging");
}


void loop() {
  float temperature = 25.5; // Example temperature
  float humidity = 60.2;    // Example humidity


  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.println(" *C");


  Serial.print("Humidity: ");
  Serial.print(humidity);
  Serial.println(" %");


  delay(5000); // Wait for 5 seconds
}
```

2. **I2C:** You have an ESP32 and two I2C sensors: a BME280 (address 0x76) and a DS3231 RTC (address 0x68). You want to read data from both. Explain how I2C addressing allows the ESP32 to communicate with each sensor individually on the same bus.

   - **Solution:**

     I2C uses a unique 7-bit (or 10-bit) address for each slave device on the bus. When the ESP32 (master) wants to communicate with a specific sensor, it first sends the sensor's unique I2C address. Only the device with that matching address will respond and participate in the subsequent data transfer. For example, to talk to the BME280, the ESP32 sends 0x76. To talk to the DS3231, it sends 0x68. This addressing scheme allows multiple

devices to share the same two physical wires (SDA and SCL) without interference.

3. **SPI:** You are designing an ESP32 project that requires both an SD card for data logging and a TFT display for user interface. Both devices use SPI. How would you connect them to the ESP32, and what is the role of the Chip Select (CS) line in this setup?

    - **Solution:**

      For SPI, both the SD card and the TFT display would share the common SCLK, MOSI, and MISO lines from the ESP32. However, each device would require its own dedicated Chip Select (CS) line. For instance, the SD card might use GPIO 5 as its CS line, and the TFT display might use GPIO 15 as its CS line. When the ESP32 wants to communicate with the SD card, it pulls the SD card's CS line LOW while keeping the TFT display's CS line HIGH. When it wants to communicate with the TFT display, it pulls the TFT's CS line LOW and the SD card's CS line HIGH. The CS line acts as an enable signal, allowing the master to select which specific slave device it wants to talk to at any given moment, preventing data collisions.

Conclusion

Understanding UART, I2C, and SPI is crucial for developing robust and versatile ESP32 projects. Each protocol has its strengths and ideal applications: UART for simple debugging and point-to-point communication, I2C for connecting multiple low-speed sensors, and SPI for high-speed peripherals like displays and memory cards. By mastering these communication methods, you can effectively integrate a wide array of components into your embedded systems.

References

[1] ESP32 Official Documentation: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/uart.html

[2] I2C Bus Specification: https://www.nxp.com/docs/en/user-guide/UM10204.pdf

[3] SPI Bus Wikipedia: https://en.wikipedia.org/wiki/Serial_Peripheral_Interface

[4] Adafruit BME280 Library: https://github.com/adafruit/Adafruit_BME280_Library

[5] Arduino SD Library: https://www.arduino.cc/en/reference/sd